

Blocco 5 - Gentle Introduction ai JOIN

Sintassi, cardinalità ed esempi progressivi dal modello normalizzato

Relational Databases & SQL

Materiale aggiuntivo

Perché servono i JOIN

Nel blocco 3 abbiamo separato i dati in relazioni più pulite; ora impariamo a ricomporli quando facciamo domande.

Dal blocco 3 al blocco 5

- ▶ nel blocco 3 abbiamo trasformato tabelle uniche in tabelle collegate;
- ▶ questa trasformazione riduce duplicazioni e anomalie;
- ▶ però una domanda reale spesso richiede dati distribuiti in più tabelle;
- ▶ il JOIN è l'operazione che ricompone le relazioni attraverso le chiavi;
- ▶ la difficoltà non è solo scrivere la sintassi: è controllare il significato delle righe ottenute.

Il problema concreto

Domanda informale

Per ogni riga di vendita vogliamo mostrare data vendita, cliente, città, codice prodotto, nome prodotto, quantità e prezzo applicato.

Osservazione

Nello schema normalizzato nessuna tabella contiene da sola tutte queste informazioni.

Schema di partenza

- ▶ `customers`: una riga per cliente;
- ▶ `products`: una riga per prodotto;
- ▶ `sales`: una riga per vendita;
- ▶ `sale_items`: una riga per prodotto venduto dentro una vendita.

Idea

Le chiavi sono come codici di riferimento: non copiano l'informazione, indicano dove trovarla.

Mini schema del caso vendite

```
customers(customer_id, customer_name, city)
products(product_id, sku, product_name, category)
sales(sale_id, sale_date, customer_id)
sale_items(sale_id, line_no, product_id, quantity, unit_price)
```

sales.customer_id punta a customers.customer_id. sale_items.sale_id punta a sales.sale_id.
sale_items.product_id punta a products.product_id.

Sintassi base

Un JOIN abbina righe di due tabelle quando una condizione è vera.

Forma minima

```
SELECT colonne
FROM tabella_1 AS t1
JOIN tabella_2 AS t2
  ON t2.chiave = t1.chiave_esterna;
```

ON dichiara la regola di collegamento. Senza una regola corretta, il risultato perde significato.

Come leggere la sintassi

- ▶ FROM: tabella di partenza della domanda;
- ▶ AS: alias, cioè nome breve usato nella query;
- ▶ JOIN: tabella da aggiungere;
- ▶ ON: condizione che collega le righe;
- ▶ SELECT: colonne finali, spesso prese da più tabelle.

Primo JOIN: vendita e cliente

```
SELECT
  s.sale_id,
  s.sale_date,
  c.customer_name,
  c.city
FROM sales AS s
JOIN customers AS c
  ON c.customer_id = s.customer_id
ORDER BY s.sale_id;
```

Ogni vendita ha un cliente obbligatorio. Il risultato resta una riga per vendita.

Perché usare gli alias

- ▶ rendono la query più corta;
- ▶ chiariscono da quale tabella arriva ogni colonna;
- ▶ evitano ambiguità quando due tabelle hanno colonne con lo stesso nome;
- ▶ aiutano a leggere query con più join.

Esempio mentale

`s.sale_date` si legge come “la data della vendita `s`”.

Secondo JOIN: vendita e righe vendita

```
SELECT
    s.sale_id,
    s.sale_date,
    si.line_no,
    si.product_id,
    si.quantity
FROM sales AS s
JOIN sale_items AS si
    ON si.sale_id = s.sale_id
ORDER BY s.sale_id, si.line_no;
```

Qui la granularità cambia: non più una riga per vendita, ma una riga per riga vendita.

Cardinalità

Un JOIN non aggiunge solo colonne: può cambiare il numero di righe.

Domanda da fare sempre

Dopo il join voglio una riga per cosa?

- ▶ una riga per cliente;
- ▶ una riga per vendita;
- ▶ una riga per riga vendita;
- ▶ una riga per prodotto;
- ▶ una riga per collegamento tra due entità.

- ▶ un cliente può avere molte vendite;
- ▶ una vendita può avere molte righe vendita;
- ▶ un prodotto può comparire in molte righe vendita;
- ▶ quando si entra nel lato “molti”, le righe possono moltiplicarsi;
- ▶ questa moltiplicazione si chiama spesso *fan-out*.

Controllo del fan-out

```
SELECT COUNT(*) AS sales_rows  
FROM sales;
```

```
SELECT COUNT(*) AS joined_rows  
FROM sales AS s  
JOIN sale_items AS si  
  ON si.sale_id = s.sale_id;
```

Se una vendita contiene più prodotti, il secondo conteggio sarà maggiore del primo.

Fan-out: errore o comportamento atteso?

- ▶ non è un errore se la domanda chiede una riga per riga vendita;
- ▶ è un errore se la domanda chiede una riga per vendita;
- ▶ per questo la granularità va dichiarata prima della query;
- ▶ dopo ogni join importante conviene controllare il numero di righe.

Ricostruire il report

Usiamo più JOIN per ottenere una vista leggibile senza tornare alla tabella unica.

Domanda informale

Report richiesto

Mostrare ogni prodotto venduto, con data vendita, cliente, città, SKU, nome prodotto, quantità e prezzo della riga.

Granularità corretta

Una riga per riga vendita, non una riga per vendita e non una riga per prodotto.

- ▶ tabella base logica: `sale_items`;
- ▶ aggiungere `sales` per data e cliente;
- ▶ aggiungere `customers` per nome e città;
- ▶ aggiungere `products` per SKU e nome prodotto;
- ▶ ordinare per vendita e numero riga.

Implementazione completa

```
SELECT
  s.sale_id,
  s.sale_date,
  c.customer_name,
  c.city,
  p.sku,
  p.product_name,
  si.quantity,
  si.unit_price
FROM sale_items AS si
JOIN sales AS s
  ON s.sale_id = si.sale_id
JOIN customers AS c
  ON c.customer_id = s.customer_id
JOIN products AS p
  ON p.product_id = si.product_id
ORDER BY s.sale_id, si.line_no;
```

- ▶ `sale_items` decide il denominatore: una riga per riga vendita;
- ▶ `sales` non duplica i clienti, ma porta il riferimento al cliente;
- ▶ `customers` aggiunge attributi anagrafici;
- ▶ `products` aggiunge attributi di catalogo;
- ▶ la tabella unica viene ricostruita solo come risultato della query.

INNER JOIN e LEFT JOIN

Non tutti i collegamenti hanno lo stesso significato.

Definizione pratica

INNER JOIN conserva solo le righe che trovano una corrispondenza nella tabella collegata.

- ▶ è adatto a relazioni obbligatorie;
- ▶ se manca la riga collegata, la riga di partenza scompare dal risultato;
- ▶ negli schemi con chiavi esterne obbligatorie è spesso la scelta naturale.

Definizione pratica

`LEFT JOIN` conserva tutte le righe della tabella di sinistra, anche quando non esiste corrispondenza a destra.

- ▶ è adatto a relazioni opzionali;
- ▶ le colonne della tabella destra diventano `NULL` quando il collegamento manca;
- ▶ è utile per cercare “cose senza collegamento”.

Prodotti mai venduti

```
SELECT
  p.product_id,
  p.sku,
  p.product_name
FROM products AS p
LEFT JOIN sale_items AS si
  ON si.product_id = p.product_id
WHERE si.sale_id IS NULL
ORDER BY p.product_id;
```

Partiamo da `products` perché vogliamo conservare anche i prodotti senza righe vendita.

Clienti senza vendite

```
SELECT
  c.customer_id,
  c.customer_name,
  c.city
FROM customers AS c
LEFT JOIN sales AS s
  ON s.customer_id = c.customer_id
WHERE s.sale_id IS NULL
ORDER BY c.customer_id;
```

La condizione `s.sale_id IS NULL` intercetta i clienti che non hanno trovato corrispondenza.

Errori frequenti

Molti errori sui JOIN sono errori di significato prima che errori di sintassi.

Errore: condizione di join mancante

```
-- Da evitare: produce combinazioni non motivate
SELECT s.sale_id, c.customer_name
FROM sales AS s
JOIN customers AS c
  ON true;
```

Ogni vendita verrebbe combinata con ogni cliente. Il numero di righe esplode senza significato di dominio.

Errore: colonna non qualificata

```
-- Ambiguo se molte tabelle hanno colonne simili
SELECT sale_id, customer_id
FROM sales AS s
JOIN customers AS c
  ON c.customer_id = s.customer_id;
```

Nelle query con join conviene qualificare le colonne: s.sale_id, c.customer_id.

Errore: usare LEFT JOIN come scorciatoia

- ▶ LEFT JOIN non è un join “più sicuro”;
- ▶ significa che la relazione può mancare;
- ▶ se la relazione è obbligatoria, un mancato match può segnalare dati incoerenti;
- ▶ scegliere il tipo di join è una decisione di modello, non solo di sintassi.

Costruire query per passaggi

Una query con più JOIN si scrive in modo incrementale.

- ① scrivere la domanda in forma informale;
- ② dichiarare la granularità del risultato;
- ③ scegliere la tabella base;
- ④ aggiungere un join alla volta;
- ⑤ verificare righe e significato dopo ogni passaggio;
- ⑥ solo alla fine rifinire colonne, alias e ordinamento.

Passo 1: tabella base

```
SELECT *  
FROM sale_items;
```

Serve per orientarsi: la granularità è una riga per prodotto venduto.

Passo 2: aggiungo la vendita

```
SELECT si.sale_id, si.line_no, s.sale_date, si.product_id
FROM sale_items AS si
JOIN sales AS s
  ON s.sale_id = si.sale_id;
```

Ora ogni riga vendita ha anche la data della vendita.

Passo 3: aggiungo cliente e prodotto

```
SELECT si.sale_id, si.line_no,  
       c.customer_name,  
       p.product_name,  
       si.quantity,  
       si.unit_price  
FROM sale_items AS si  
JOIN sales AS s ON s.sale_id = si.sale_id  
JOIN customers AS c ON c.customer_id = s.customer_id  
JOIN products AS p ON p.product_id = si.product_id;
```

La query finale nasce come estensione controllata di query più piccole.

Esercizi guidati

Ogni esercizio parte da una richiesta informale, poi viene tradotto in specifica e soluzione.

Esercizio 1: problema informale

Richiesta

Il responsabile vendite vuole un elenco leggibile delle righe vendita, con cliente e prodotto.

Specifica

Una riga per riga vendita. Usare solo collegamenti obbligatori.

Esercizio 1: soluzione

```
SELECT
  s.sale_id,
  c.customer_name,
  p.sku,
  p.product_name,
  si.quantity
FROM sale_items AS si
JOIN sales AS s ON s.sale_id = si.sale_id
JOIN customers AS c ON c.customer_id = s.customer_id
JOIN products AS p ON p.product_id = si.product_id
ORDER BY s.sale_id, si.line_no;
```

Esercizio 2: problema informale

Richiesta

Il catalogo vuole sapere quali prodotti sono presenti in anagrafica ma non compaiono in nessuna vendita.

Specifica

Una riga per prodotto non venduto. Conservare tutti i prodotti e cercare il mancato collegamento.

Esercizio 2: soluzione

```
SELECT p.product_id, p.sku, p.product_name
FROM products AS p
LEFT JOIN sale_items AS si
  ON si.product_id = p.product_id
WHERE si.sale_id IS NULL
ORDER BY p.product_id;
```

Checklist finale

- ① Ho dichiarato la granularità del risultato?
- ② La tabella base corrisponde alla granularità desiderata?
- ③ Ogni JOIN ha una condizione ON motivata da chiavi o regole di dominio?
- ④ So spiegare se il join è obbligatorio o opzionale?
- ⑤ Ho controllato il fan-out quando entro nel lato molti?
- ⑥ Ho qualificato le colonne con alias leggibili?

- ▶ un join ricompone relazioni normalizzate senza tornare a duplicare i dati;
- ▶ prima di scrivere la sintassi bisogna sapere una riga per cosa si vuole ottenere;
- ▶ `INNER JOIN` conserva solo corrispondenze, `LEFT JOIN` conserva la tabella di sinistra;
- ▶ ogni moltiplicazione di righe deve essere prevista e verificata.